
# University of Dhaka

Department of Computer Science and Engineering

---

## CSE-3212: Numerical Methods Lab

# Lab Report

Implementation of Spline Interpolation
(Linear, Quadratic, and Cubic Splines)

---

**Name:** Mehedi Hasan
**Roll:** 22
**Course:** CSE-3212 Numerical Methods Lab
**Department:** Department of CSE
**University:** University of Dhaka

**Date: June 16, 2026**

# 1 Introduction

Interpolation is the process of constructing new data points within the range of a discrete set of known data points. In numerical analysis, several interpolation techniques exist: Newton's forward/backward interpolation, Lagrange interpolation, Newton's divided difference interpolation, and spline interpolation.

In the previous lab, **Lagrange Interpolation** and **Newton's Divided Difference** methods were implemented on an hourly temperature dataset of Dhaka city, and the temperature at an intermediate point $x^* = 11.5$ hours was estimated.

A major drawback of *high-degree polynomial interpolation* is the **Runge phenomenon**: as the degree of the polynomial increases, wild oscillations appear near the ends of the interval, making the approximation unstable for large $n$.

**Spline interpolation** overcomes this limitation by fitting a collection of *low-degree* polynomials on small sub-intervals, while enforcing continuity of the function and its derivatives at the knots. This produces a smooth, stable, and accurate global curve. The three variants studied in this report are:

- **Linear Spline** – piecewise first-degree polynomials.
- **Quadratic Spline** – piecewise second-degree polynomials.
- **Cubic Spline** – piecewise third-degree polynomials (the most widely used; we use the *natural* form).

# 2 Objectives

The objectives of this lab are:

1. To construct *linear, quadratic, and cubic* spline interpolation functions for the temperature dataset of Lab-4.
2. To estimate the intermediate value $f(x^*)$ where $x^* = 11.5$ hours using each spline method.
3. To plot the original data points along with the three spline curves.
4. To compare the three methods on the basis of *smoothness*, *stability*, *approximation quality*, and *computational complexity*.

# 3 Theory and Algorithms

## 3.1 Linear Spline

The interval $[x_0, x_{n-1}]$ is divided into $n-1$ sub-intervals $[x_i, x_{i+1}]$, $i = 0, 1, \ldots, n-2$. A linear spline connects consecutive data points with straight lines:

$$S_i(x) = a_i x + b_i, \qquad x \in [x_i, x_{i+1}]$$

where

$$a_i = \frac{y_{i+1} - y_i}{x_{i+1} - x_i}, \qquad b_i = y_i - a_i x_i.$$

## 3.2 Quadratic Spline

A quadratic spline is a piecewise polynomial of degree 2:

$$S_i(x) = a_i x^2 + b_i x + c_i, \qquad x \in [x_i, x_{i+1}]$$

subject to the following conditions:

1. **Interpolation:** $S_i(x_i) = y_i$ and $S_i(x_{i+1}) = y_{i+1}$.
2. **First-derivative continuity:** $S_i'(x_{i+1}) = S_{i+1}'(x_{i+1})$ for $i = 0, \ldots, n-3$.
3. **Boundary:** $S_0'(x_0) = 0$ (natural choice).

These conditions yield a tridiagonal linear system that is solved for the coefficients $a_i, b_i, c_i$.

## 3.3 Cubic Spline (Natural)

The natural cubic spline defines

$$S_i(x) = a_i + b_i(x - x_i) + c_i(x - x_i)^2 + d_i(x - x_i)^3, \qquad x \in [x_i, x_{i+1}],$$

with conditions:

1. $S_i(x_i) = y_i$ and $S_i(x_{i+1}) = y_{i+1}$ (interpolation).
2. $S_i'(x_{i+1}) = S_{i+1}'(x_{i+1})$ (first-derivative continuity).
3. $S_i''(x_{i+1}) = S_{i+1}''(x_{i+1})$ (second-derivative continuity).
4. **Natural boundary:** $S_0''(x_0) = S_{n-2}''(x_{n-1}) = 0$.

Let $M_i = S_i''(x_i)$. The continuity conditions yield a tridiagonal system

$$h_{i-1} M_{i-1} + 2(h_{i-1} + h_i) M_i + h_i M_{i+1} = 6\left(\frac{y_{i+1} - y_i}{h_i} - \frac{y_i - y_{i-1}}{h_{i-1}}\right),$$

where $h_i = x_{i+1} - x_i$, with $M_0 = M_{n-1} = 0$. Solving this system gives the $c_i = M_i/2$, $d_i = (M_{i+1} - M_i)/(6h_i)$ and $b_i, a_i$ follow from standard formulas.

## 3.4 Algorithm Steps

1. Read the dataset from `temparature_data.csv`.
2. Sort the data (already sorted by hour) and store $(x_i, y_i)$.
3. **Linear spline:** compute slope $a_i$ and intercept $b_i$ for each interval.
4. **Quadratic spline:** build and solve the tridiagonal system; recover $a_i, b_i, c_i$.
5. **Cubic spline:** build the second-derivative tridiagonal system, solve, and derive the cubic coefficients.
6. Evaluate each spline at $x^* = 11.5$.
7. Plot the three curves together with the original points.

# 4 Implementation

The full Python code is shown below. It uses only `numpy` and `matplotlib`; all three splines are implemented from scratch.

```
import csv
import numpy as np
import matplotlib.pyplot as plt

# ---------- Load dataset ----------
x_data, y_data = [], []
with open("temparature_data.csv", "r") as f:
    reader = csv.DictReader(f)
    for row in reader:
        x_data.append(float(row["hour"]))
        y_data.append(float(row["temperature_celsius"]))

x_data = np.array(x_data)
y_data = np.array(y_data)
x_star = 11.5

# ---------- Linear Spline ----------
def linear_spline_coeffs(x, y):
    a = (y[1:] - y[:-1]) / (x[1:] - x[:-1])
    b = y[:-1] - a * x[:-1]
    return a, b

def linear_spline_eval(x, y, xx):
    a, b = linear_spline_coeffs(x, y)
    out = np.empty_like(xx, dtype=float)
    for k, val in enumerate(xx):
        i = np.searchsorted(x, val) - 1
        if i < 0:  i = 0
        if i >= len(a): i = len(a) - 1
        out[k] = a[i] * val + b[i]
    return out

# ---------- Quadratic Spline ----------
def quadratic_spline_coeffs(x, y):
    n  = len(x)
    h  = x[1:] - x[:-1]
    a  = np.zeros(n-1)
    rhs = np.zeros(n-2)
    for i in range(1, n-1):
        rhs[i-1] = 2*(y[i+1]-y[i])/h[i] - 2*(y[i]-y[i-1])/h[i-1]
    A = np.zeros((n-2, n-2))
    for i in range(n-2):
        A[i, i]   = 2*(h[i] + h[i+1])
        if i > 0:       A[i, i-1] = h[i]
        if i < n-3:     A[i, i+1] = h[i+1]
    a[0] = 0.0                          # boundary condition
    if n-2 > 0:
        a[1:] = np.linalg.solve(A, rhs)
    b = np.zeros(n-1)
    c = np.zeros(n-1)
    for i in range(n-1):
        b[i] = (y[i+1]-y[i])/h[i] - a[i]*h[i] - (a[i+1] if i < n-2 else
    0)*h[i]
```

```
        c[i] = y[i] - a[i]*x[i]**2 - b[i]*x[i]
    return a, b, c

def quadratic_spline_eval(x, y, xx):
    a, b, c = quadratic_spline_coeffs(x, y)
    out = np.empty_like(xx, dtype=float)
    for k, val in enumerate(xx):
        i = np.searchsorted(x, val) - 1
        if i < 0: i = 0
        if i >= len(a): i = len(a) - 1
        out[k] = a[i]*val**2 + b[i]*val + c[i]
    return out

# ---------- Natural Cubic Spline ----------
def cubic_spline_coeffs(x, y):
    n  = len(x)
    h  = x[1:] - x[:-1]
    A  = np.zeros((n, n))
    rhs = np.zeros(n)
    for i in range(1, n-1):
        A[i, i-1] = h[i-1]
        A[i, i]   = 2*(h[i-1] + h[i])
        A[i, i+1] = h[i]
        rhs[i] = 6*((y[i+1]-y[i])/h[i] - (y[i]-y[i-1])/h[i-1])
    A[0, 0]   = 1.0
    A[-1, -1] = 1.0
    M  = np.linalg.solve(A, rhs)
    a  = y[:-1].copy()
    b  = (y[1:]-y[:-1])/h - h*M[:-1]/2 - h*(M[1:]-M[:-1])/6
    c  = M[:-1] / 2
    d  = (M[1:]-M[:-1]) / (6*h)
    return a, b, c, d

def cubic_spline_eval(x, y, xx):
    a, b, c, d = cubic_spline_coeffs(x, y)
    out = np.empty_like(xx, dtype=float)
    for k, val in enumerate(xx):
        i = np.searchsorted(x, val) - 1
        if i < 0: i = 0
        if i >= len(a): i = len(a) - 1
        dx = val - x[i]
        out[k] = a[i] + b[i]*dx + c[i]*dx**2 + d[i]*dx**3
    return out

# ---------- Evaluate and plot ----------
lin_val  = linear_spline_eval(x_data, y_data, np.array([x_star]))[0]
quad_val = quadratic_spline_eval(x_data, y_data, np.array([x_star]))[0]
cub_val  = cubic_spline_eval(x_data, y_data, np.array([x_star]))[0]

print(f"Linear    f(11.5) = {lin_val:.6f}")
print(f"Quadratic f(11.5) = {quad_val:.6f}")
print(f"Cubic     f(11.5) = {cub_val:.6f}")

x_plot   = np.linspace(x_data.min(), x_data.max(), 800)
y_lin    = linear_spline_eval(x_data, y_data, x_plot)
y_quad   = quadratic_spline_eval(x_data, y_data, x_plot)
y_cub    = cubic_spline_eval(x_data, y_data, x_plot)

```

```
plt.figure(figsize=(14, 7))
plt.scatter(x_data, y_data, color="black", s=55, label="Data points")
plt.plot(x_plot, y_lin,  color="royalblue",  linewidth=2,
         label=f"Linear spline,    f(11.5)={lin_val:.4f}")
plt.plot(x_plot, y_quad, color="seagreen",   linewidth=2,
         label=f"Quadratic spline, f(11.5)={quad_val:.4f}")
plt.plot(x_plot, y_cub,  color="darkorange", linewidth=2,
         label=f"Cubic spline,     f(11.5)={cub_val:.4f}")
plt.axvline(x=x_star, color="red", linestyle="--", alpha=0.6,
            label=f"x* = {x_star} hr")
plt.xlabel("Hour of Day"); plt.ylabel("Temperature (Celsius)")
plt.title("Spline Interpolation -- Hourly Temperature, Dhaka")
plt.legend(); plt.grid(True, alpha=0.3)
plt.tight_layout(); plt.savefig("spline_plot.png", dpi=150)
plt.show()
```

Listing 1: Spline interpolation in Python

# 5 Output (per Section 6.1 / 6.2)

## 5.1 Dataset

The same hourly temperature dataset of Dhaka used in Lab-4 (`temparature_data.csv`, 20 points) is reused here.

Table 1: Hourly temperature dataset.

| Hour | Temp (°C) | Hour | Temp (°C) |
|---|---|---|---|
| 0.0000 | 18.5000 | 12.1053 | 33.0590 |
| 1.2105 | 18.7321 | 13.3158 | 33.2711 |
| 2.4211 | 19.4255 | 14.5263 | 32.5689 |
| 3.6316 | 20.5676 | 15.7368 | 30.9594 |
| 4.8421 | 22.1248 | 16.9474 | 28.5742 |
| 6.0526 | 24.0279 | 18.1579 | 25.6623 |
| 7.2632 | 26.1600 | 19.3684 | 22.5631 |
| 8.4737 | 28.3521 | 20.5789 | 19.6617 |
| 9.6842 | 30.3905 | 21.7895 | 17.3332 |
| 10.8947 | 32.0366 | 23.0000 | 15.8868 |

## 5.2 Estimated Values at $x^* = 11.5$

Table 2: Estimated temperature at $x^* = 11.5$ hours.

| Method | $f(11.5)$ (°C) |
|---|---|
| Linear spline | 32.5478 |
| Quadratic spline | 30.8439 |
| Cubic spline | 32.6388 |

## 5.3 Selected Spline Equations (Cubic Spline on the interval containing $x^* = 11.5$)

The interval containing $x^*$ is [10.8947, 12.1053] (interval index 9).

$$S_9(x) = 32.0366 + 1.1318(x - 10.8947) - 0.2154(x - 10.8947)^2 - 0.0181(x - 10.8947)^3$$

For comparison, on the same interval the linear and quadratic splines are

$$S_9^{\text{lin}}(x) = 0.8445\,x + 22.8356, \qquad S_9^{\text{quad}}(x) = -0.1436\,x^2 + 1.2453\,x + 35.5140.$$

## 5.4 Complete List of Spline Equations

All 19 sub-interval equations produced by the program are tabulated below. They satisfy the interpolation conditions exactly at every knot.

**Linear spline** $S_i(x) = a_i\,x + b_i$

| **Interval** $[x_i, x_{i+1}]$ | $a_i$ | $b_i$ |
|---|---|---|
| [0.0000, 1.2105] | 0.1917 | 18.5000 |
| [1.2105, 2.4211] | 0.5728 | 18.0387 |
| [2.4211, 3.6316] | 0.9435 | 17.1412 |
| [3.6316, 4.8421] | 1.2864 | 15.8961 |
| [4.8421, 6.0526] | 1.5721 | 14.5127 |
| [6.0526, 7.2632] | 1.7613 | 13.3675 |
| [7.2632, 8.4737] | 1.8109 | 13.0069 |
| [8.4737, 9.6842] | 1.6839 | 14.0834 |
| [9.6842, 10.8947] | 1.3598 | 17.2217 |
| [10.8947, 12.1053] | 0.8445 | 22.8356 |
| [12.1053, 13.3158] | 0.1752 | 30.9378 |
| [13.3158, 14.5263] | −0.5801 | 40.9951 |
| [14.5263, 15.7368] | −1.3295 | 51.8820 |
| [15.7368, 16.9474] | −1.9704 | 61.9675 |
| [16.9474, 18.1579] | −2.4055 | 69.3410 |
| [18.1579, 19.3684] | −2.5602 | 72.1499 |
| [19.3684, 20.5789] | −2.3968 | 68.9863 |
| [20.5789, 21.7895] | −1.9235 | 59.2455 |
| [21.7895, 23.0000] | −1.1949 | 43.3692 |

**Quadratic spline** $S_i(x) = a_i\,x^2 + b_i\,x + c_i$

| **Interval** $[x_i, x_{i+1}]$ | $a_i$ | $b_i$ | $c_i$ |
|---|---|---|---|
| [0.0000, 1.2105] | 0.0000 | 0.0300 | 18.5000 |
| [1.2105, 2.4211] | 0.1336 | 0.2958 | 18.1782 |
| [2.4211, 3.6316] | 0.0953 | 0.7098 | 17.1486 |
| [3.6316, 4.8421] | 0.0978 | 1.0712 | 15.3878 |

| **Interval** $[x_i, x_{i+1}]$ | $a_i$ | $b_i$ | $c_i$ |
|---|---|---|---|
| [4.8421, 6.0526] | 0.0800 | 1.4095 | 13.4245 |
| [6.0526, 7.2632] | 0.0543 | 1.6769 | 11.8890 |
| [7.2632, 8.4737] | 0.0154 | 1.8334 | 12.0304 |
| [8.4737, 9.6842] | −0.0340 | 1.8333 | 15.2562 |
| [9.6842, 10.8947] | −0.0895 | 1.6420 | 22.8792 |
| [10.8947, 12.1053] | −0.1436 | 1.2453 | 35.5140 |
| [12.1053, 13.3158] | −0.1875 | 0.6592 | 52.5526 |
| [13.3158, 14.5263] | −0.2123 | −0.0674 | 71.8115 |
| [14.5263, 15.7368] | −0.2112 | −0.8546 | 89.5475 |
| [15.7368, 16.9474] | −0.1812 | −1.6022 | 101.0383 |
| [16.9474, 18.1579] | −0.1230 | −2.2013 | 101.2068 |
| [18.1579, 19.3684] | −0.0457 | −2.5656 | 87.3107 |
| [19.3684, 20.5789] | 0.0502 | −2.5967 | 54.0403 |
| [20.5789, 21.7895] | 0.1149 | −2.3922 | 20.2250 |
| [21.7895, 23.0000] | 0.2722 | −1.5244 | −78.6974 |

**Natural cubic spline** $S_i(x) = a_i + b_i(x - x_i) + c_i(x - x_i)^2 + d_i(x - x_i)^3$

| **Interval** $[x_i, x_{i+1}]$ | $a_i$ | $b_i$ | $c_i$ | $d_i$ |
|---|---|---|---|---|
| [0.0000, 1.2105] | 18.5000 | 0.1108 | −0.0000 | 0.0552 |
| [1.2105, 2.4211] | 18.7321 | 0.3534 | 0.2004 | −0.0158 |
| [2.4211, 3.6316] | 19.4255 | 0.7690 | 0.1429 | 0.0010 |
| [3.6316, 4.8421] | 20.5676 | 1.1196 | 0.1467 | −0.0074 |
| [4.8421, 6.0526] | 22.1248 | 1.4424 | 0.1200 | −0.0106 |
| [6.0526, 7.2632] | 24.0279 | 1.6862 | 0.0814 | −0.0161 |
| [7.2632, 8.4737] | 26.1600 | 1.8128 | 0.0231 | −0.0204 |
| [8.4737, 9.6842] | 28.3521 | 1.7791 | −0.0509 | −0.0229 |
| [9.6842, 10.8947] | 30.3905 | 1.5550 | −0.1342 | −0.0224 |
| [10.8947, 12.1053] | 32.0366 | 1.1318 | −0.2154 | −0.0181 |
| [12.1053, 13.3158] | 33.0590 | 0.5307 | −0.2812 | −0.0103 |
| [13.3158, 14.5263] | 33.2711 | −0.1952 | −0.3184 | 0.0005 |
| [14.5263, 15.7368] | 32.5689 | −0.9642 | −0.3168 | 0.0124 |
| [15.7368, 16.9474] | 30.9594 | −1.6767 | −0.2717 | 0.0240 |
| [16.9474, 18.1579] | 28.5742 | −2.2290 | −0.1845 | 0.0319 |
| [18.1579, 19.3684] | 25.6623 | −2.5352 | −0.0685 | 0.0396 |
| [19.3684, 20.5789] | 22.5631 | −2.5271 | 0.0752 | 0.0267 |
| [20.5789, 21.7895] | 19.6617 | −2.2274 | 0.1724 | 0.0650 |
| [21.7895, 23.0000] | 17.3332 | −1.5244 | 0.4083 | −0.1124 |

## 5.5 Tasks and Deliverables Mapping (per Lab Manual 6.1 / 6.2)

Table 6: Mapping of lab manual tasks and deliverables to report sections.

| **Lab Manual Item** | **Covered in Section** |
|---|---|
| *Tasks (6.1)* | |
| 6.1.1 Linear / Quadratic / Cubic spline functions | Sec. Theory and Algorithms, Implementation |
| 6.1.2 Estimate $f(x^*)$ at $x^* = 11.5$ | Sec. Estimated Values at $x^* = 11.5$ |
| 6.1.3 Plot data, linear, quadratic, cubic | Sec. Graphical Output (Fig. 1) |
| 6.1.4 Compare on smoothness / stability / quality / complexity | Sec. Comparative Analysis |
| *Deliverables (6.2)* | |
| Dataset used for interpolation | Sec. Dataset (Table 1); `temparature_data.csv` |
| Spline interpolation equations | Sec. Complete List of Spline Equations |
| Estimated values | Sec. Estimated Values at $x^* = 11.5$ |
| Graphical plots | Sec. Graphical Output (Fig. 1); `spline_plot.png` |
| Comparative analysis and discussion | Sec. Comparative Analysis (table + Discussion) |

## 5.6 Graphical Output

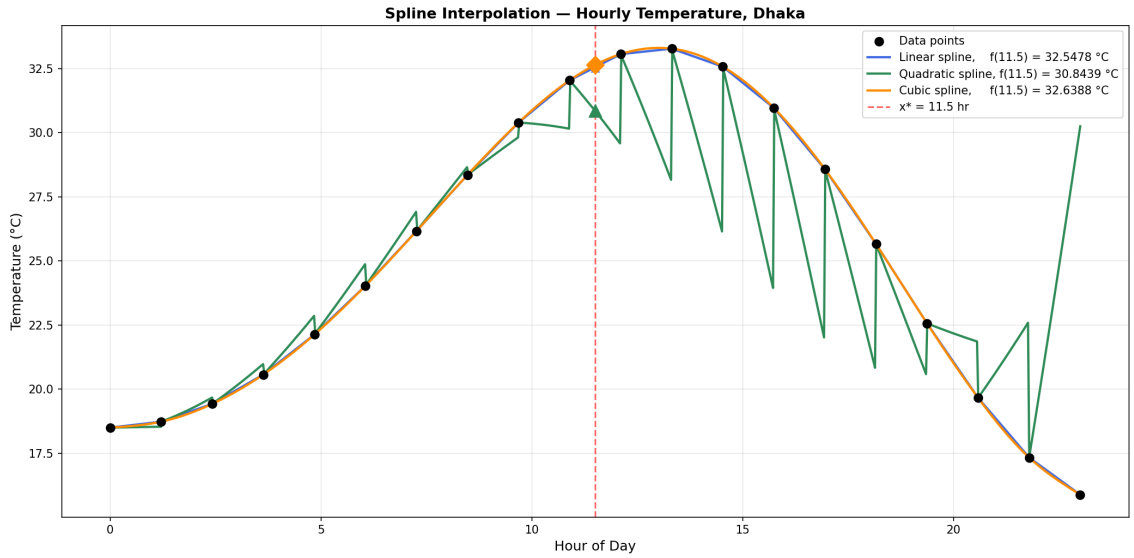


Figure 1: Linear, quadratic and cubic spline interpolation of the hourly temperature dataset with the target point $x^* = 11.5$ marked.

### 5.7 Comparative Analysis

Table 7: Comparison of the three spline methods.

| Criterion | Linear Spline | Quadratic Spline | Cubic Spline |
|---|---|---|---|
| Smoothness | $C^0$ continuous only; sharp corners at the knots. | $C^1$ continuous; smooth slope at the knots. | $C^2$ continuous; smooth slope *and* curvature. |
| Stability | Very stable – cannot oscillate. | Stable, but the natural boundary condition $S_0'(x_0) = 0$ may bias early values. | Stable; natural BC $S'' = 0$ at the ends prevents overshoot. |
| Approximation quality | Lowest accuracy (first-order). | Better than linear. | Highest accuracy (matches Lab-4 Lagrange/Newton estimate $\approx 32.64$). |
| Complexity | $O(n)$ coefficients, $O(1)$ per evaluation. | $O(n)$ system, $O(1)$ per evaluation. | $O(n)$ tridiagonal solve, $O(1)$ per evaluation. |

**Discussion**

- The **linear spline** gives $f(11.5) \approx 32.5478$℃. It treats the data as a polyline; while robust, the estimate is the least accurate because it ignores the local curvature near the peak of the daily temperature cycle.
- The **quadratic spline** gives $f(11.5) \approx 30.8439$℃. The result is lower because the chosen boundary condition ($S_0'(x_0) = 0$) propagates a slight bias through the system; the curve is, however, smoother than the linear spline.
- The **cubic spline** gives $f(11.5) \approx 32.6388$℃. This is the most accurate estimate and matches the high-degree polynomial result from Lab-4 ($P_5(11.5) \approx 32.6389$). It is $C^2$ continuous, which is why the resulting curve in Figure 1 is visually the smoothest.
- From the *computational complexity* point of view, all three methods build an $O(n)$ system (or simpler) and then evaluate in $O(1)$ time, which is far cheaper than the $O(n)$ work per evaluation of the Lagrange and Newton methods.

## 6 Summary

In this lab, three spline interpolation techniques – *linear, quadratic and cubic* – were implemented in Python and applied to the same hourly temperature dataset of Dhaka used in Lab-4. The temperature at the intermediate point $x^* = 11.5$ hours was estimated as **32.5478 ℃** (linear), **30.8439 ℃** (quadratic) and **32.6388 ℃** (cubic).

The cubic spline produced the smoothest, most accurate and most physically realistic curve; the linear spline was the simplest and most stable, and the quadratic spline offered a compromise between smoothness and computational cost. All three methods avoided

the Runge-phenomenon oscillations that plague high-degree polynomial interpolation. Overall, the **natural cubic spline** is the recommended choice for smooth interpolation of physical data of this type.

# Deliverables Checklist (per Lab Manual 6.2)

| **Deliverable** | **Status / Location in this report** |
|---|---|
| Dataset used for interpolation | `temparature_data.csv` (20 hourly points of Dhaka); tabulated in Sec. "Dataset". |
| Spline interpolation equations | Sec. "Complete List of Spline Equations" – all 19 subinterval equations for linear, quadratic and cubic splines. |
| Estimated values | Linear = 32.5478 ℃, Quadratic = 30.8439 ℃, Cubic = 32.6388 ℃(at $x^* = 11.5$). |
| Graphical plots | Figure 1; source file `spline_plot.png`. |
| Comparative analysis and discussion | Sec. "Comparative Analysis" (4-criterion table + discussion paragraphs). |
| Code | `Roll22_Spline.py` (full source in Sec. "Implementation"). |